

CrashGuard: Handlebar-Mounted Crash Detection and Emergency Alert System

Implementation, Modelling, Dataset Audit, and Wokwi Simulation

Adheesh Garg (23BCT0122)

Aastha Gupta (23BCT0082)

Ayan Gatani (23BCT0059)

Programming for IoT Boards Lab (BCSE312P)

Class VL2026270103137 Faculty: Syamasudha Veeragandham (22727)

Assessment 5, Review 2 28 August 2026

<https://github.com/qwerty-dvorak/crashguard-iot>

Abstract

This report converts the approved CrashGuard proposal into an ESP32 prototype with a repeatable experimental workflow while separating measurements from assumptions. The system uses a handlebar-mounted MPU-6050, an ESP32 DevKit V1, a local light and sound alarm, a cancel button, and an optional Blynk event sent over a phone hotspot. No physical hardware was available for this review. Consequently, the evidence consists of source-level unit tests, exact C++ replay of the firmware detector over a deterministic physics-informed dataset, an audit of an independent powered-two-wheeler dataset, diagram validation, and an executable Wokwi project. The generated evaluation contains 675 ten-second events, or 675,000 six-axis samples at 100 Hz. CrashGuard v2 produced 114 true positives, 525 true negatives, 0 false positives, and 36 false negatives: sensitivity 76.0% and specificity 100.0%. These are simulation results, not claims of road-crash performance. The original Review-1 rule was too strict, and the supplied base paper contains reproducibility and numerical inconsistencies. Both are therefore treated as inputs to be tested, not final truth. All firmware, Wokwi wiring, scenarios, dataset acquisition and generation scripts, recorded hashes, analysis code, derived results, and paper sources are delivered with this report.

Research integrity statement. No physical test, Wokwi cloud run, battery-runtime measurement, network delivery measurement, or real road crash is represented as completed. A Wokwi CI token is an external credential and is not committed. The reported classification results come only from deterministic replay and are labelled accordingly.

Contents

1	Problem, scope, and contributions	4
1.1	Assessment traceability	4
2	Critical audit of the supplied base paper	4
3	Related research and design position	5
4	System architecture	6
5	Complete electrical design	6
5.1	Pin mapping and electrical rationale	8
5.2	Analytic power budget, not a measurement	9

6	Datasets, provenance, and processing	9
6.1	Dataset inventory	9
6.2	Physics-informed generated dataset	9
7	Mathematical decision model	10
7.1	Activity gate	10
7.2	State rules	10
8	Baseline, original method, and proposed algorithm	11
9	Implementation and software verification	12
10	Wokwi simulation	12
11	Results	13
11.1	Deterministic event replay	13
11.2	Independent PTW feature audit	15
12	Validity, safety, privacy, and limitations	15
13	Experiment reproduction and verification	16
14	Conclusion and next validation stage	16
A	Repository map	17
B	Data manifest	17
C	Firmware excerpt	17

List of Figures

1	Operational architecture. Detection and cancellation remain local; network availability affects delivery, not sensing.	6
2	Complete deployable circuit. The Wokwi diagram substitutes a virtual low-current piezo directly on GPIO25, while physical hardware uses the shown NPN driver. . . .	7
3	Event-level sensitivity and specificity with Wilson 95% intervals. Intervals quantify finite simulation-event counts, not real-world uncertainty.	13
4	One reproducible example each of pothole, parked tip-over, and low-side crash. Dashed lines are v2 thresholds. A threshold crossing alone does not decide the class.	14
5	Peak features after anti-aliased resampling of the public PTW data. Red bars are the four published fall intervals; blue bars are seven extreme manoeuvre recordings. . . .	15

List of Tables

1	Mapping from the Review-2 brief to evidence in this package.	4
2	What is transferred from the base paper and what is not.	5
3	Research comparison. Reported scores are not directly comparable because vehicles, placements, labels, and protocols differ.	6
4	Final ESP32 pin map.	8
5	Physical prototype bill of materials and critical selection rules.	8
6	Data used or audited in this review.	9
7	CrashGuard v2 decision parameters.	11
8	Automated Wokwi scenarios and acceptance criteria.	12

9	Event-level replay on 150 positive and 525 negative simulated events. Median delay is from the method's first candidate to confirmation.	13
---	--	----

1 Problem, scope, and contributions

India recorded 172,890 road deaths in 2023; 77,539, or 44.8%, were users of two-wheelers according to the latest detailed MoRTH report available at the time of this review [1]. CrashGuard addresses one narrow part of this problem: reducing the interval between an inertial event and an alert. It does not prevent a crash, assess injury, guarantee communication, or replace emergency services.

The technical question is whether a low-cost, vehicle-mounted six-axis IMU can reject common high-dynamic riding events while retaining sensitivity to fall-like signatures. Three research questions guide the work:

- RQ1:** Does temporal correlation of impact, rotation, and persistent rest reduce false triggers relative to an acceleration-only rule?
- RQ2:** Do the original Review-1 thresholds survive comparison with an independent powered-two-wheeler dataset?
- RQ3:** Can the same production C++ decision logic execute on an ESP32, under host replay, and in Wokwi without a simulation-only sensor branch?

The contribution is not a claim of a finished safety product. It is a falsifiable prototype package with: (i) a corrected electrical design, (ii) a state-machine detector with centralized parameters, (iii) a public dataset audit, (iv) a deterministic robustness dataset, (v) three algorithm comparisons, and (vi) automated firmware-level Wokwi scenarios.

1.1 Assessment traceability

Table 1: Mapping from the Review-2 brief to evidence in this package.

Required item	Delivered evidence	Primary files or sections
Complete circuit	Power, sensor, GPIO, actuator driver, ADC divider, and charging constraint	Section 5 ; <code>wokwi/diagram.json</code>
Dataset or collection	Two downloaded archives, processed external PTW recordings, and generated event data	Section 6 ; <code>data/</code>
Mathematical model	Equations, parameters, state transitions, and timing rules	Section 7 ; <code>crash_detector.h</code>
Algorithm comparison	Acceleration baseline, Review-1 rule, and v2 state machine	Sections 8 and 11
Working simulation	Wokwi circuit plus crash, rejection, and cancel automation scenarios	Section 10 ; <code>wokwi/</code>
Code implementation	Production ESP32 sketch, portable detector, unit tests, and exact replay executable	Section 9 ; <code>analysis/</code>

2 Critical audit of the supplied base paper

The supplied paper by Tian et al. [2] is a useful methodological reference, but it solves a different sensing problem. Its device is attached to a person’s shin, samples an LSM6DSM at 50 Hz, segments 170-sample windows, and classifies human activities and falls with a CNN-transformer. CrashGuard is fixed to a motorcycle handlebar, samples an MPU-6050 at 100 Hz, and detects vehicle dynamics. Mounting location changes the signal-generating body, reference orientation, vibration spectrum, and meaning of a fall label. Directly copying its network or its metrics would therefore be a category error.

Table 2: What is transferred from the base paper and what is not.

Aspect	Valid lesson	CrashGuard decision
Six-axis time series	Acceleration alone does not encode the temporal sequence adequately	Correlate acceleration, angular rate, orientation, and post-event rest
Class imbalance	Accuracy can hide misses when the minority event is rare	Report TP, TN, FP, FN, sensitivity, specificity, precision, and F1
Long temporal context	Pre-event and post-event evidence matter	Use a small deterministic state machine instead of a 3.4 s tensor model
Hardware co-design	Model choice must respect memory, power, and latency	Use constant-memory arithmetic suitable for the ESP32
Reported human-fall score	Useful only for its stated dataset and placement	Do not use 93.1% as motorcycle evidence

The paper and accompanying archive were audited rather than accepted at face value. Four issues matter for replication:

1. It reports an ROC operating point $(\text{FPR}, \text{TPR}) = (0.055, 0.905)$ and AUC 0.768. Any nondecreasing ROC curve passing through that point has the lower bound

$$\text{AUC} \geq \text{TPR}(1 - \text{FPR}) = 0.905(1 - 0.055) = 0.855, \quad (1)$$

even if it contributes no area before the point. The published AUC and operating point cannot both describe the same valid ROC curve.

2. It calls the deployment representation both “8-bit” and “Q7.8”. Fixed-point Q notation varies, but Q7.8 requires more than eight total bits under conventional definitions. An exact sign, integer, fraction, saturation, and rounding specification is absent.
3. The paper reports 9,195 and 42,351 windows, class ratios, augmentation, and subject-level evaluation. The downloaded HealthLINK archive contains raw recording files but no complete machine-readable label, window, split, and augmentation manifest sufficient to reconstruct those two tensors exactly.
4. Its 502 kB reported model footprint, FPGA-specific resource claims, and human-body placement do not establish feasibility or validity on an ESP32 handlebar unit.

The archive is retained and hashed for provenance, but is not mixed into the motorcycle evaluation. This is a deliberate exclusion based on domain and label integrity, not an attempt to discard inconvenient data.

3 Related research and design position

The strongest directly relevant lineage is Boubezoul et al. Their 2013 detector used powered-two-wheeler dynamics [3], and their later data article released four controlled fall recordings and seven critical or extreme manoeuvre recordings [4]. Gelmini et al. developed and experimentally validated vehicle-mounted crash logic and later a novel two-wheeler algorithm [5, 6]. Mahariba et al. used the Boubezoul data for a machine learning study [7]. Lee et al. compared threshold, supervised, and unsupervised IMU approaches for e-scooter pre-impact fall detection [8]. The 2026 VZCrash release is much larger, with real fleet crashes, but its commercial four-wheel vehicle population does not remove the powered-two-wheeler domain gap [9].

Table 3: Research comparison. Reported scores are not directly comparable because vehicles, placements, labels, and protocols differ.

Work	Platform and data	Method	Relevance or limitation
Tian et al., 2026	Shin device; 50 Hz; human falls	CNN plus multi-head attention	Strong data and metric lessons; wrong sensing body for this system; audit issues in Section 2
Boubezoul et al., 2013/2019	Instrumented PTW; 1 kHz; four falls and seven extreme records	Rules based on vehicle dynamics	Closest public raw data; small event count and limited sensor ranges
Gelmini et al., 2019/2021	Two-wheeled vehicle tests	Engineered crash algorithm and validation	Directly supports staged vehicle-dynamic evidence; implementation is not this ESP32 prototype
Mahariba et al., 2022	Public PTW inertial recordings	Conventional machine learning	Demonstrates reuse of the PTW dataset; dataset scale remains small
Lee et al., 2024	E-scooter, 60 Hz, 20 participants	Threshold, supervised, and autoencoder models	Supports multi-feature thresholds and shows that a small rule can compete in a controlled protocol
VZCrash, 2026	73,010 commercial vehicles, 100 Hz IMU	Threshold and deep learning benchmarks	Scale and real-crash realism are valuable; vehicle and mounting domains differ
This work	Handlebar model; 100 Hz; 675 simulated events plus 11 public PTW records	Activity-gated, symmetric impact-rotation correlation, tilted-rest verification	Exact deployable rule and open replay; no physical validation

The literature does not justify selecting a complex model merely because one is available. A supervised network would require a sufficiently large, representative, labelled dataset from the target motorcycle placement and a subject or vehicle independent split. That dataset is not available here. The current rule is transparent, constant-memory, testable sample by sample, and appropriate for the evidence level. ML remains a later comparison after physical data collection.

4 System architecture

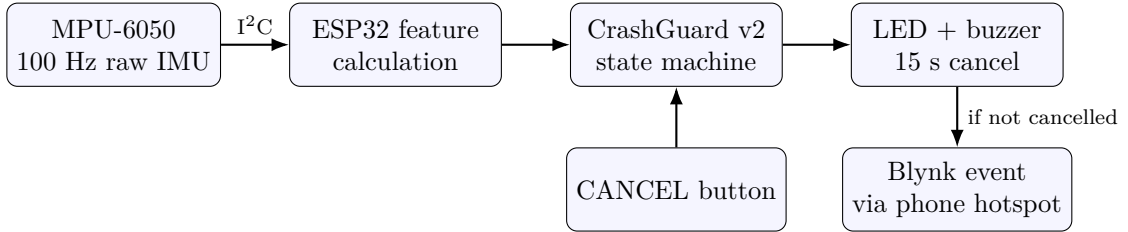


Figure 1: Operational architecture. Detection and cancellation remain local; network availability affects delivery, not sensing.

The firmware attempts one Blynk event after a confirmed event remains uncanceled for 15 s. Current Blynk device events can trigger notifications and the device HTTPS API accepts an event code and description [10]. The phone hotspot supplies Internet connectivity only. It does not automatically expose the phone’s GPS position to ESP32 code. Therefore, this implementation explicitly sends “Location unavailable”. A location claim requires either a GNSS receiver or a separately developed and consented companion application.

5 Complete electrical design

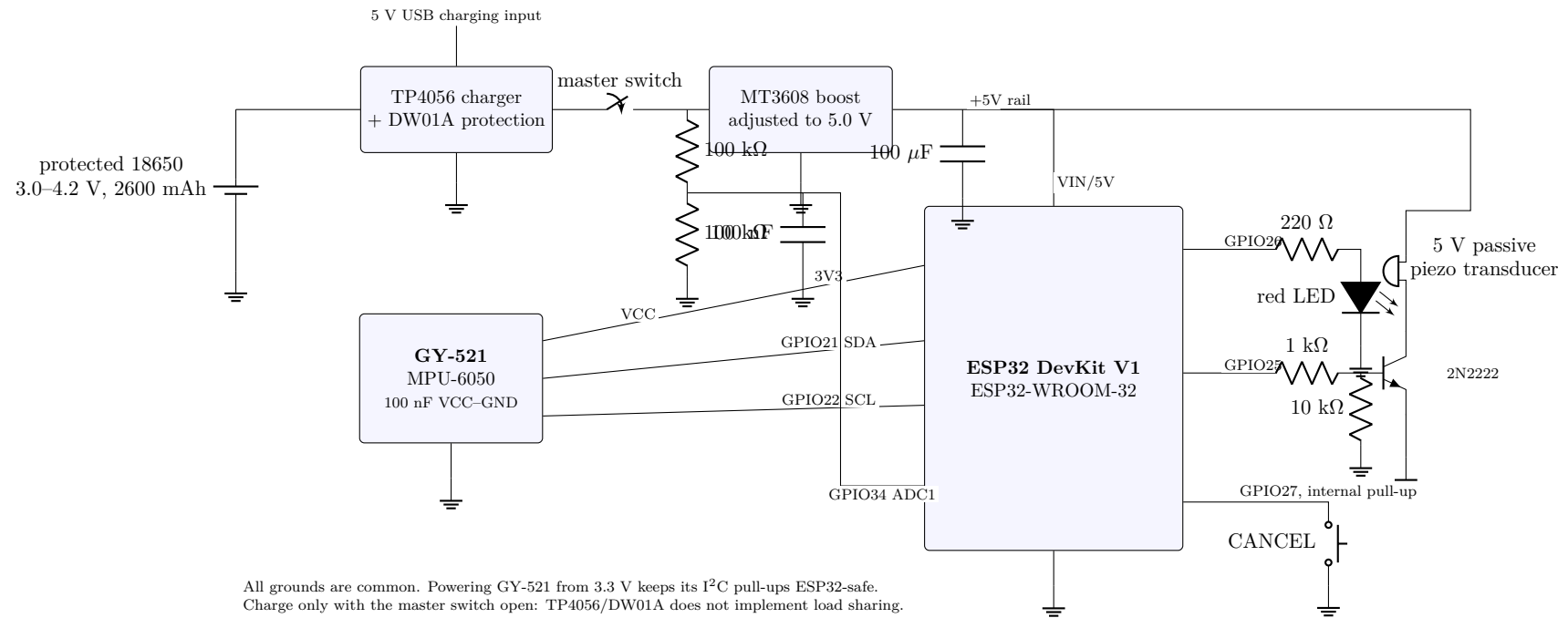


Figure 2: Complete deployable circuit. The Wokwi diagram substitutes a virtual low-current piezo directly on GPIO25, while physical hardware uses the shown NPN driver.

5.1 Pin mapping and electrical rationale

Table 4: Final ESP32 pin map.

ESP32 pin	Direction	Connection	Reason
GPIO21	Bidirectional	MPU-6050 SDA	Default I ² C data; 3.3 V pull-ups
GPIO22	Output	MPU-6050 SCL	400 kHz clock
GPIO26	Output	220 Ω plus red LED	Local visual alarm
GPIO25	Output	1 k Ω to 2N2222 base	Isolates buzzer current from GPIO
GPIO27	Input, pull-up	Button to ground	Active-low cancel with no floating input
GPIO34	Input only, ADC1	100 k Ω /100 k Ω divider	Battery reading remains available while Wi-Fi uses ADC2 resources
3V3	Power	GY-521 VCC	Prevents 5 V I ² C pull-ups
VIN/5V	Power	MT3608 5.0 V output	Adequate headroom for DevKit regulator

Table 5: Physical prototype bill of materials and critical selection rules.

Part	Qty.	Required rating or function
ESP32 DevKit V1, ESP32-WROOM-32	1	5 V VIN, 3.3 V GPIO, Wi-Fi station
GY-521 MPU-6050 module	1	Six-axis IMU, powered at 3.3 V
Protected 18650 cell	1	2600 mAh from a traceable vendor; never use an unprotected salvaged cell
TP4056 plus DW01A module	1	Single-cell charging and protection; charge with load switched off
MT3608 boost module	1	Adjust and verify 5.0 V before connecting ESP32
5 V passive piezo transducer	1	At most 30 mA, driven at 2.4 kHz
2N2222 NPN transistor	1	Low-side sounder driver
Red LED and momentary button	1 each	Visual alarm and active-low CANCEL input
Resistors	5	220 Ω , 1 k Ω , 10 k Ω , and two 100 k Ω
Capacitors	3	100 μ F bulk and two 100 nF local/ADC capacitors
Master switch, insulated enclosure, mount	1 each	Load isolation, strain relief, and fixed handlebar orientation

No display was proposed or added: local status is conveyed by LED and sound, while event history appears in Blynk. Adding a display solely to satisfy a generic checklist would increase load and weather sealing complexity without serving the approved architecture.

The MPU-6050 is configured to $\pm 16g$ and $\pm 2000^\circ \text{s}^{-1}$ so impact and rapid rotation do not immediately saturate the narrower default ranges [11]. The 100 nF sensor capacitor and 100 μ F rail capacitor reduce local supply transients. GPIO34 is input-only, which is appropriate for the divider, and its ADC1 assignment avoids the ESP32 ADC2 and Wi-Fi conflict documented for the platform [12].

The audible part is a 5 V passive piezo transducer rated at no more than 30 mA. GPIO25 supplies a 2.4 kHz waveform through a 1 k Ω base resistor; the 2N2222 switches transducer current and a 10 k Ω resistor keeps it off during boot. A magnetic sounder would require a flyback diode and is not interchangeable with the specified piezo part.

The TP4056/DW01A module is a charger with cell protection, not a true power path. The specified circuit must be switched off while charging. A future simultaneous charge-and-run version

must replace it with a load-sharing power path charger. This corrects a common but unsafe schematic shortcut.

5.2 Analytic power budget, not a measurement

An engineering estimate uses a 5 V average rail current of about 120 mA in connected standby. Thus $P_{5V} \approx 0.60$ W. At 3.7 V and assumed 90% boost efficiency,

$$I_{cell} \approx \frac{0.60}{3.7(0.90)} = 0.180 \text{ A.} \quad (2)$$

If 90% of a 2600 mAh cell is usable, the idealized runtime is $2.34/0.180 \approx 13$ h. ESP32 radio duty cycle, converter efficiency, cell age, temperature, alarm duty, and regulator loss can shift this result substantially. It is a sizing calculation awaiting a logged discharge test.

6 Datasets, provenance, and processing

6.1 Dataset inventory

Table 6: Data used or audited in this review.

Dataset	Size used	Inputs and format	Role and target
HealthLINK archive [13]	123,821,562-byte ZIP; 41 raw recording files	Six-axis human IMU CSV/XLSX, source sampling described as 50 Hz	Base-paper provenance audit only; no reproducible window label/split manifest found
Boubezoul PTW [4]	11 recordings: four falls, seven manoeuvres	Time, three accelerations in m/s^2 , three angular rates in degree/s; tab-separated CSV at 1 kHz	Independent feature sanity check; target is fall versus critical manoeuvre
CrashGuard simulated	675 events, 675,000 rows	Event ID, binary label, scenario, time, six IMU channels; gzip CSV at 100 Hz	Exact firmware replay; 150 fall-like positive and 525 non-crash negative events

The external 1 kHz PTW channels are resampled to 100 Hz with a polyphase anti-alias filter before feature extraction. The four fall time stamps are taken from the dataset article; a three-second interval centered on each is analysed. Complete manoeuvre recordings are retained as negatives. The published sensor ranges are narrower than the MPU-6050 configuration, and the acceleration appears unsuitable as a persistent raw gravity reference. For that reason the external audit tests only impact and rotation gates, not the complete tilted-rest state machine.

6.2 Physics-informed generated dataset

Each event lasts 10 s and uses seed 20260828. The generator constructs roll $\phi(t)$, pitch $\theta(t)$, yaw $\psi(t)$, body-frame gravity, linear manoeuvre pulses, sensor bias, and Gaussian noise. With roll and pitch, the ideal gravity vector in sensor coordinates is

$$\mathbf{g}_b(t) = \begin{bmatrix} -\sin \theta(t) \\ \sin \phi(t) \cos \theta(t) \\ \cos \phi(t) \cos \theta(t) \end{bmatrix}. \quad (3)$$

Linear components are added for potholes, bumps, braking, turning, impact, vertical unloading, and post-impact ringing. Angular rates are numerical derivatives of the attitude trajectories. Finally, a per-event bias and sample noise are added and values are clipped to $\pm 16g$ and $\pm 2000^\circ/\text{s}$. This is more realistic than arbitrary threshold crossings, but it is still a model whose assumptions may favor the proposed logic.

Nine balanced-within-scenario groups contain 75 events each: normal riding, pothole, speed breaker, hard brake, sharp turn, parked tip-over, impact then recovery, low-side crash, and high-side crash. Only the last two are positive. There is no training set because no statistical model is fitted. Parameters are read directly from the C++ header, and the evaluation script does not optimize them. Nevertheless, threshold selection and simulator development occurred in the same project, so the resulting scores are an exploratory robustness check, not an independent generalization estimate.

The gzip timestamp is fixed to zero and its embedded filename is empty. The compressed file is therefore byte-reproducible, not merely value-reproducible. SHA-256 provenance can be checked with:

```
sha256sum -c data/manifest.sha256
```

7 Mathematical decision model

For raw signed values r_a, r_ω under the selected full-scale ranges, firmware conversion is

$$a_i = \frac{r_{a,i}}{2048} g, \quad \omega_i = \frac{r_{\omega,i}}{16.4} ^\circ/\text{s}. \quad (4)$$

At each 10 ms sample, resultant acceleration and angular rate are

$$A_k = \sqrt{a_x^2 + a_y^2 + a_z^2}, \quad \Omega_k = \sqrt{\omega_x^2 + \omega_y^2 + \omega_z^2}. \quad (5)$$

During a stationary two-second startup calibration, 200 samples establish the reference gravity vector $\mathbf{r}$. Calibration is rejected unless $0.85 \leq \|\mathbf{r}\| \leq 1.15g$ and gyroscope RMS is at most $5^\circ/\text{s}$. Tilt relative to the installed upright pose is

$$\Theta_k = \cos^{-1} \left(\text{clip} \frac{\mathbf{a}_k \cdot \mathbf{r}}{\|\mathbf{a}_k\| \|\mathbf{r}\|}, -1, 1 \right). \quad (6)$$

7.1 Activity gate

A parked motorcycle can be knocked over and should not automatically generate a remote alert. Define sample activity as

$$U_k = [|A_k - 1| \geq 0.08] \vee [\Omega_k \geq 4]. \quad (7)$$

A leaky evidence accumulator increases by Δt when U_k is true and decays by $\Delta t/2$ otherwise, clipped to $[0, 1000]$ ms. Reaching 1000 ms qualifies a ride and opens a 5000 ms rolling activity memory. This heuristic does not prove that the engine is running; it only prevents a single isolated parked tip from qualifying itself.

7.2 State rules

The parameter values below are compiled into both firmware and replay code.

Table 7: CrashGuard v2 decision parameters.

Parameter	Value	Meaning
A_H	2.00 g	High-impact threshold
A_L	0.35 g	Unloading or near-free-fall threshold
Ω_T	90 °/s	Rotation threshold
Θ_T	55°	Final tilt threshold
$[A_{R,L}, A_{R,H}]$	[0.70, 1.30] g	Rest acceleration band
Ω_R	30 °/s	Maximum angular rate at rest
T_C	750 ms	Impact-rotation correlation window
T_R	2000 ms	Continuous tilted-rest hold
T_V	6000 ms	Candidate verification timeout
T_X	15000 ms	Rider cancel interval

For an activity-qualified sample, an impact candidate is

$$H_k = [A_k \geq A_H] \vee [A_k \leq A_L]. \quad (8)$$

Rotation may precede or follow impact. The detector accepts the pair when H_k occurs within T_C of a previous $\Omega \geq \Omega_T$, or a qualifying rotation occurs within T_C after H_k . This symmetric correlation corrects the Review-1 assumption that impact must always arrive first.

The final evidence is continuous tilted rest:

$$R_k = [\Theta_k \geq \Theta_T] \wedge [0.70 \leq A_k \leq 1.30] \wedge [\Omega_k \leq 30]. \quad (9)$$

Only if R_k remains true for T_R does the system enter the countdown. Failure to establish rest by T_V rejects the candidate. Countdown expiry produces one remote-event attempt; pressing CANCEL returns to idle. All time subtractions use unsigned wraparound-safe arithmetic.

8 Baseline, original method, and proposed algorithm

The comparison deliberately includes both a weak baseline and the actual Review-1 proposal.

Acceleration-only baseline:

classify an event positive if any $A_k \geq 2g$. It is fast and sensitive but ignores context.

Review-1 ordered logic:

require $A_k \geq 4g$, then $\Omega_k \geq 200^\circ/\text{s}$ within 500 ms, then $\Theta_k \geq 60^\circ$ with $0.7 \leq A_k \leq 1.3g$ for 3 s.

CrashGuard v2:

activity gating, high or low acceleration, symmetric temporal correlation with rotation, calibrated 3D tilt, two-second rest verification, and local cancellation.

Listing 1: Executable-level pseudocode for CrashGuard v2.

```

1 sample MPU6050 at 100 Hz
2 compute acceleration magnitude, angular-rate magnitude, and calibrated tilt
3 update leaky ride-activity evidence
4
5 IDLE:
6   if ride memory is active and (A >= 2.0 g or A <= 0.35 g):
7     open candidate
8     if rotation >= 90 deg/s occurred in the previous 750 ms:
9       go to VERIFY_REST
10    else:
11      go to CORRELATE
12 CORRELATE:
13   if rotation >= 90 deg/s within 750 ms: go to VERIFY_REST
14   else if 750 ms expires: reject and return to IDLE

```

```

15 VERIFY_REST:
16   require tilt >= 55 deg, 0.70 <= A <= 1.30 g,
17     and rotation <= 30 deg/s continuously for 2000 ms
18   if satisfied: sound local alarm and start 15000 ms countdown
19   if 6000 ms candidate timeout expires: reject
20 COUNTDOWN:
21   if CANCEL is pressed: silence and return to IDLE
22   if countdown expires: send one Blynk event and enter ALERT_SENT

```

9 Implementation and software verification

`wokwi/sketch.ino` is production-oriented code. It configures the MPU-6050 registers directly, uses 400 kHz I²C, maintains a 100 Hz deadline, calibrates gyroscope bias and gravity reference, debounces an active-low button edge, drives the local alarm, reads the protected-cell divider, and optionally calls Blynk over HTTPS. It has no conditional branch that injects synthetic sensor values.

The decision implementation lives in `wokwi/crash_detector.h` and has no Arduino dependency. `analysis/detector_replay.cpp` includes that same header, ensuring that reported v2 decisions are made by the deployment logic rather than a Python reimplementaion. Four native unit tests cover rotation before impact, pothole rejection, parked tip rejection, and cancel. The complete pipeline compiles the tests with warnings as errors, runs them, generates the dataset, streams the gzip CSV through the C++ replay program, calculates metrics, audits external data, and renders the PDF figures.

An ESP32 Arduino core 3.3.11 target compile with all compiler warnings enabled succeeds. The offline firmware image uses 392,947 bytes, or 29% of the 1,310,720-byte application partition, and statically allocated globals use 29,808 bytes, or 9% of 327,680 bytes. These compiler figures establish build feasibility but do not measure maximum stack, heap fragmentation, timing, or energy on a running board. Arduino CLI requires the primary `.ino` name to match its directory, so a four-line adapter under `wokwi/arduino-build/sketch/` includes the root Wokwi `sketch.ino`; it does not duplicate firmware logic.

Cloud credentials are held in an optional untracked `secrets.h`. The offline build remains fully functional. Failed Wi-Fi or HTTPS delivery does not block sampling. A production revision should add a persistent outbound queue, delivery acknowledgement, TLS trust-store policy, watchdog recovery, and rate limiting.

10 Wokwi simulation

Wokwi requires the filename `diagram.json`; the commonly mistyped `daigram.json` will not be discovered. The supplied diagram contains an ESP32 DevKit V1, MPU-6050, LED and resistor, buzzer, button, and a potentiometer that emulates the battery divider. Its pin map matches [Table 1](#) and [figure 2](#). Wokwi’s MPU control values are in g and degree/s, so scenario values map directly to firmware units [14]. The official diagram and automation formats are used [15, 16].

Table 8: Automated Wokwi scenarios and acceptance criteria.

Scenario	Stimulus	Required observation
Crash	Qualifying motion, 240 °/s rotation, 5 g impact, then a tilted 1 g rest pose	CRASH_CONFIRMED, followed after 15 s by ALERT_SENT
Pothole	Qualifying motion and 5 g pulse without rotation	CANDIDATE_REJECTED; GPIO26 remains low
Cancel	Same fall signature, then active-low cancel press during countdown	ALERT_CANCELLED; GPIO26 returns low and no alert is due

The Wokwi diagram was linted with Wokwi CLI 0.26.1 with no errors. Its only informational result is that the widely used `wokwi-esp32-devkit-v1` part identifier is undocumented and could change. The source is compiled locally for the ESP32 target. Cloud scenario execution requires the user’s Wokwi CI token. The scenario files are executable acceptance tests, but this report does not invent a passing cloud run in the absence of that credential. Wokwi also does not model cell chemistry, TP4056 charging behavior, boost-converter efficiency, mechanical mounting, waterproofing, or real vibration.

11 Results

11.1 Deterministic event replay

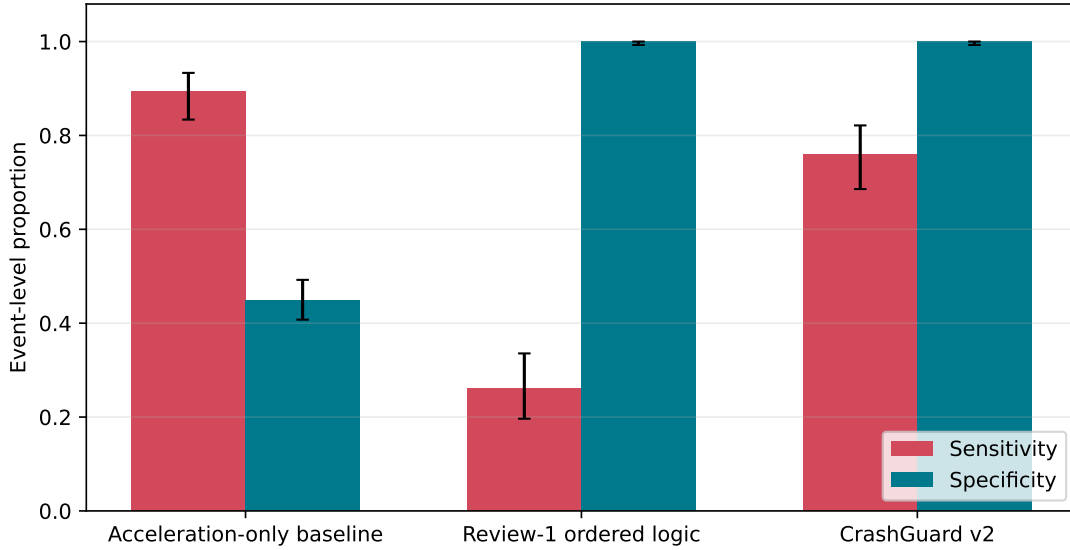


Figure 3: Event-level sensitivity and specificity with Wilson 95% intervals. Intervals quantify finite simulation-event counts, not real-world uncertainty.

Table 9: Event-level replay on 150 positive and 525 negative simulated events. Median delay is from the method’s first candidate to confirmation.

Method	TP	TN	FP	FN	Sens.	Spec.	F1
Acceleration only	134	236	289	16	89.3%	45.0%	0.468
Review-1 ordered	39	525	0	111	26.0%	100.0%	0.413
CrashGuard v2	114	525	0	36	76.0%	100.0%	0.864

CrashGuard v2 sensitivity has Wilson 95% interval [68.6%, 82.1%]. With zero observed false positives among 525 negatives, specificity is 100%, but its lower Wilson bound is only 99.3%. Therefore “zero in this run” is not equivalent to “false alarms are impossible”. The median v2 confirmation delay is 2310 ms, close to the explicit two-second rest requirement. Review-1 median confirmation delay is 3120 ms because of its three-second rest hold.

The acceleration-only rule demonstrates the expected tradeoff: high sensitivity but 289 false triggers, mostly high-dynamic road events. The Review-1 rule has no false positives but misses 111 of 150 simulated crashes. V2 recovers 75 of those missed positives without adding a false positive in this generated set. All 75 high-side events are detected; 39 of 75 lower dynamic low-side events are detected. These 36 low-side misses are the main known failure mode and make the original 90% detection objective unmet at this evidence stage.

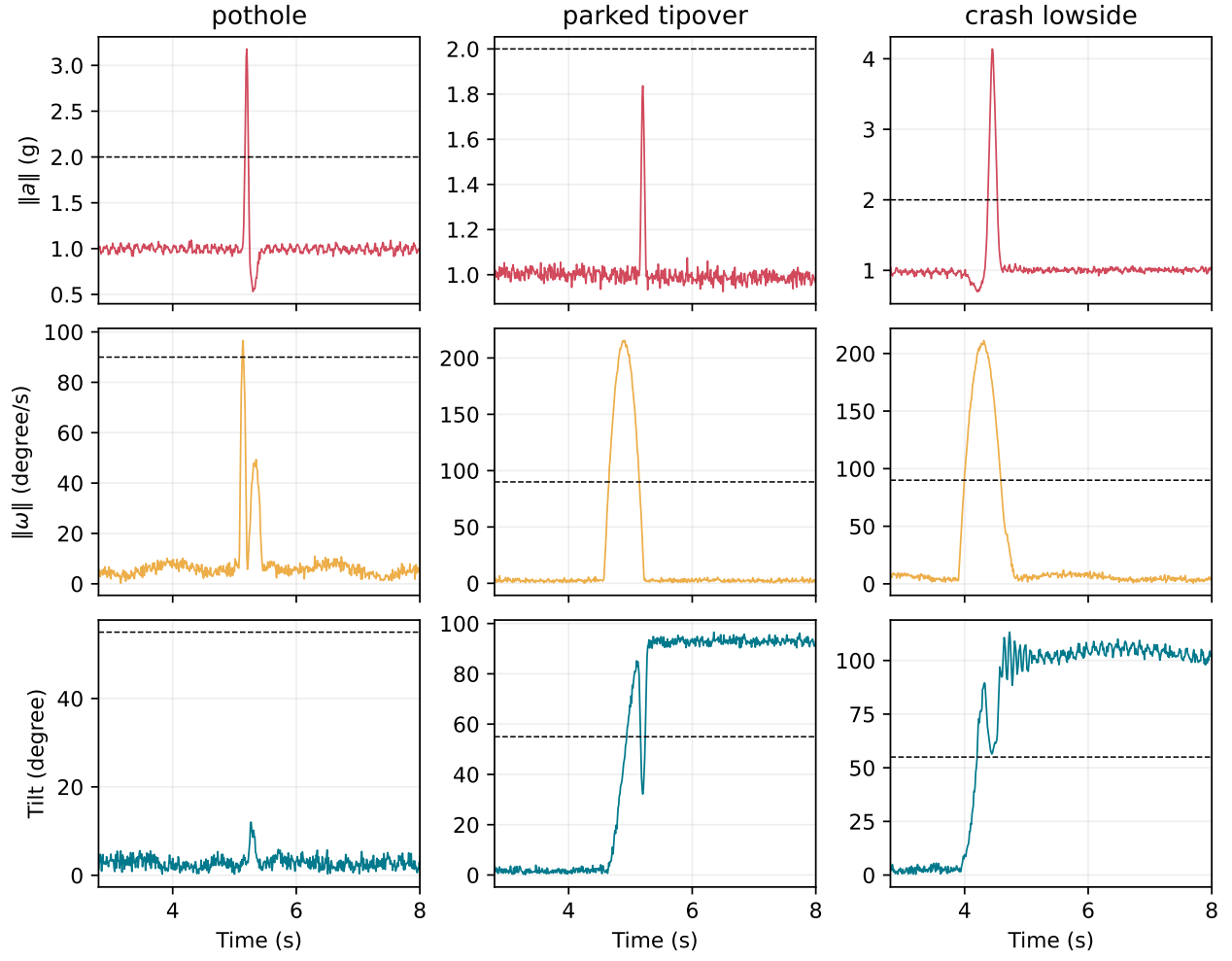


Figure 4: One reproducible example each of pothole, parked tip-over, and low-side crash. Dashed lines are v_2 thresholds. A threshold crossing alone does not decide the class.

11.2 Independent PTW feature audit

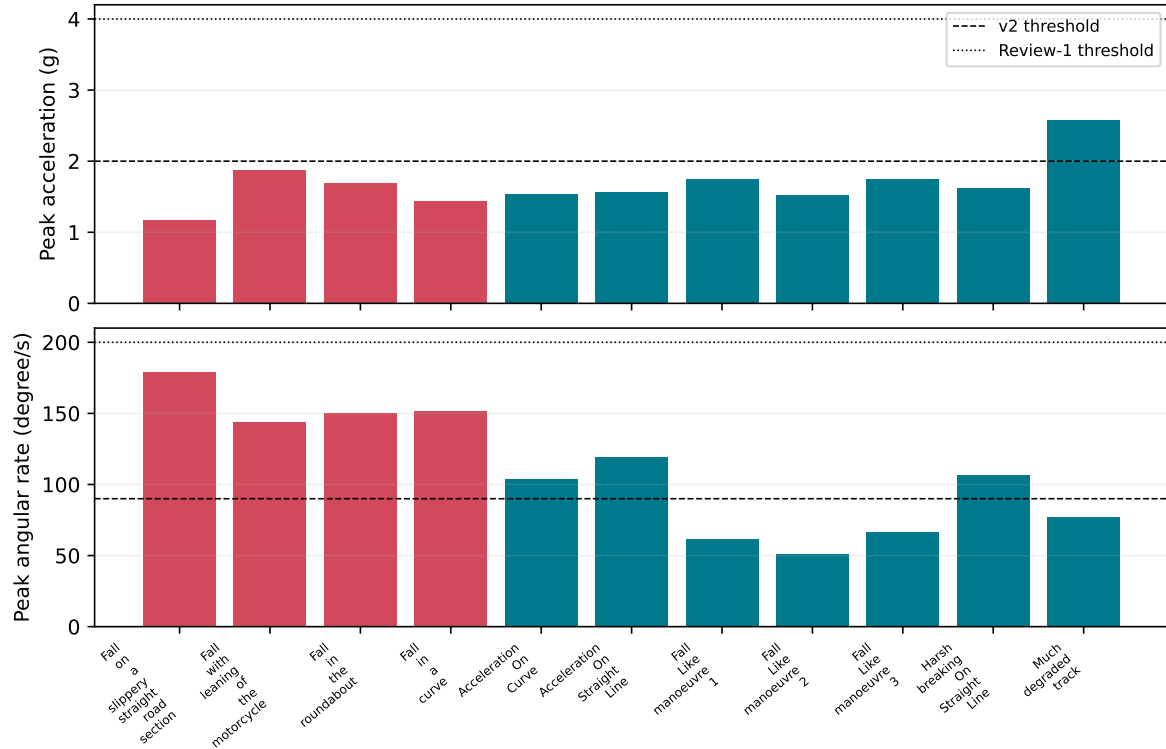


Figure 5: Peak features after anti-aliased resampling of the public PTW data. Red bars are the four published fall intervals; blue bars are seven extreme manoeuvre recordings.

None of the four external fall intervals reaches the Review-1 4 g impact or 200 °/s rotation thresholds: 0/4 and 0/4 pass, respectively. In contrast, 4/4 pass the v2 high-or-low acceleration gate and 4/4 pass its 90 °/s rotation gate. This is evidence that the original thresholds were not portable to the closest public data. It is not a full v2 accuracy result because the archive’s acceleration representation does not support the firmware’s calibrated gravity and rest test. Several non-fall manoeuvres also cross one v2 gate, which reinforces the need for temporal and rest evidence rather than proving the final classifier.

12 Validity, safety, privacy, and limitations

1. **Simulation-to-road gap.** Generated trajectories omit frame flex, mounting slip, engine harmonics, impacts at varied locations, temperature drift, sensor clipping details, and rider-vehicle separation. Scores cannot be promoted as field accuracy.
2. **Dataset dependence.** The external PTW set has only four controlled falls. HealthLINK has the wrong sensing domain, and its paper-level processing could not be exactly reconstructed.
3. **Threshold development bias.** The simulation generator and detector were developed in the same project. No blinded physical holdout has been evaluated.
4. **No measured non-detection exposure.** Event specificity is not a false-alert rate per hour or kilometre. A road exposure study is required for the operational objective.
5. **Communication.** A hotspot may be disabled, out of coverage, or disconnected. HTTPS success is not emergency-contact receipt.
6. **Location and privacy.** No coordinates are fabricated. Adding GNSS or a companion application creates consent, retention, and access control requirements.
7. **Safety.** Intentional public-road crash tests are prohibited. Future controlled tip-over work must use an unmanned frame, remote release, soft containment, cell protection, and responsible lab supervision.

This prototype is not a certified emergency, medical, or automotive safety device. A false negative may prevent an alert and a false positive may alarm contacts. The local cancel interval is a usability control, not a guarantee.

13 Experiment reproduction and verification

Clone the public repository with GitHub CLI, then run every command from its root. The `sudo` command uses Void Linux package names:

```
gh repo clone qwerty-dvorak/crashguard-iot
cd crashguard-iot

sudo xbps-install -S base-devel python3 python3-pip arduino-cli curl unzip \
    texlive texlive-most texlive-latexmk

python3 -m venv .venv
. .venv/bin/activate
python -m pip install -r analysis/requirements.txt
sh data/download_datasets.sh
./analysis/run_all.sh
sha256sum -c data/manifest.sha256

arduino-cli config add board_manager.additional_urls \
    https://espressif.github.io/arduino-esp32/package_esp32_index.json
arduino-cli core update-index
arduino-cli core install esp32:esp32
(cd wokwi && arduino-cli compile --fqbn esp32:esp32:esp32 \
    --build-path build arduino-build/sketch)

latexmk -pdf report.tex
```

To lint and run Wokwi scenarios after obtaining a CI token:

```
curl -L https://wokwi.com/ci/install.sh | sh
export PATH="$HOME/bin:$PATH"
(cd wokwi && wokwi-cli lint .)
export WOKWI_CLI_TOKEN='wok_replace_with_your_token'
(cd wokwi && wokwi-cli . --scenario scenario-crash.yaml \
    --timeout 25000 --expect-text ALERT_SENT)
(cd wokwi && wokwi-cli . --scenario scenario-pothole.yaml \
    --timeout 10000 --expect-text CANDIDATE_REJECTED)
(cd wokwi && wokwi-cli . --scenario scenario-cancel.yaml \
    --timeout 12000 --expect-text ALERT_CANCELLED)
```

14 Conclusion and next validation stage

The implementation satisfies the Review-2 deliverables at simulation level and exposes rather than hides what remains unknown. The original 4 g, 200 °/s ordered method does not survive the public PTW audit and has only 26.0% sensitivity on generated events. The proposed activity-gated symmetric state machine raises simulated sensitivity to 76.0%, keeps zero observed false positives in 525 generated negative events, and uses the exact code deployed in the ESP32 sketch. It still misses 36 lower-dynamic events and has no road-valid accuracy claim.

The next valid experiment is staged, not a real crash. First, verify rail voltages, sensor orientation, buzzer transistor current, charge-only interlock, and sampling jitter on the bench. Second, collect labelled stationary tip-overs and suspended-frame drops onto a soft mat. Third, log at least 300 km of ordinary riding across potholes, bumps, braking, turns, and mounting variations. Freeze thresholds before a held-out route, report alerts per hour and per kilometre with confidence bounds, and compare against a small interpretable ML model only after that target-domain dataset exists.

A Repository map

Path	Purpose
report.tex, references.bib	This paper and its bibliography
wokwi/sketch.ino	Production ESP32 firmware
wokwi/crash_detector.h	Portable exact state-machine implementation
wokwi/diagram.json	Wokwi circuit description
wokwi/scenario-*.yaml	Automated crash, rejection, and cancel stimuli
analysis/generate_simulated_dataset.py	Deterministic physics-informed generator
analysis/detector_replay.cpp	Exact firmware detector replay and baselines
analysis/evaluate.py	Metrics, Wilson intervals, external audit, and figures
analysis/test_detector.cpp	Native state-machine regression tests
data/download_datasets.sh	Idempotent source-archive downloader, verifier, and PTW extractor
data/manifest.sha256	Source and generated archive hashes
results/summary.json	Machine-readable configuration, metrics, and external features

B Data manifest

```
# Downloaded source archives. Verify with: sha256sum -c data/manifest.sha256
0938d21a89475cbba2e377496b71862c8147e78e21974972e3ca2e0beef95b18 data/raw/
HealthLINK_Falls_Dataset.zip
76d9e3171ead05147bbd38de194e8d34ba800614c98d38836a8bfca9ab329497 data/raw/
PMC6660605_SupplementaryFiles.zip

# Generated by analysis/run_all.sh with seed 20260828.
27c6d5a17fd55fd56688d0881d03698efd596549d112fd087b9d06d9ab0a9fc7 data/generated/simulated_imu.csv.gz
```

C Firmware excerpt

The complete source is supplied separately so it can compile directly. The following excerpt shows the real sampling path and demonstrates that replay values are not injected into firmware.

```
1 uint32_t nextSampleUs = 0;
2 bool lastButtonPressed = false;
3 float gyroOffsetX = 0.0f;
4 float gyroOffsetY = 0.0f;
5 float gyroOffsetZ = 0.0f;
6
7 bool writeRegister(uint8_t reg, uint8_t value) {
8     Wire.beginTransaction(kMpuAddress);
9     Wire.write(reg);
10    Wire.write(value);
11    return Wire.endTransmission(true) == 0;
12 }
13
14 bool readRaw(crashguard::Sample &sample) {
15     Wire.beginTransaction(kMpuAddress);
16     Wire.write(0x3B);
```

```

17 if (Wire.endTransmission(false) != 0) return false;
18 if (Wire.requestFrom(kMpuAddress, static_cast<uint8_t>(14), true) != 14) {
19     return false;
20 }
21
22 auto readI16 = []() -> int16_t {
23     return static_cast<int16_t>((Wire.read() << 8) | Wire.read());
24 };
25
26 const int16_t rawAx = readI16();
27 const int16_t rawAy = readI16();
28 const int16_t rawAz = readI16();
29 (void)readI16(); // Temperature is not used by the detector.
30 const int16_t rawGx = readI16();
31 const int16_t rawGy = readI16();
32 const int16_t rawGz = readI16();
33
34 sample.timeMs = millis();
35 sample.axG = rawAx / kAccelLsbPerG;
36 sample.ayG = rawAy / kAccelLsbPerG;
37 sample.azG = rawAz / kAccelLsbPerG;
38 sample.gxDps = rawGx / kGyroLsbPerDps - gyroOffsetX;
39 sample.gyDps = rawGy / kGyroLsbPerDps - gyroOffsetY;
40 sample.gzDps = rawGz / kGyroLsbPerDps - gyroOffsetZ;
41 return true;
42 }
43
44 bool configureMpu6050() {
45     Wire.begin(kSdaPin, kSclPin, 400000);
46     delay(100);

```

References

- [1] Ministry of Road Transport and Highways, Government of India, “Road accidents in india 2023,” Transport Research Wing, Tech. Rep., 2025. [Online]. Available: <https://morth.nic.in/sites/default/files/Road-Accident-in-India-2023-Publications.pdf>
- [2] J. Tian, S. Xu, P. Gerstoft, M. Sarkar, A. A. Moore, P. Mercier, and C. Paolini, “A multi-head attention transformer model for wearable in situ fall detection,” *IEEE Access*, vol. 14, pp. 85 242–85 262, 2026.
- [3] A. Boubezoul, S. Espié, B. Larnaudie, and S. Bouaziz, “A simple fall detection algorithm for powered two wheelers,” *Control Engineering Practice*, vol. 21, no. 3, pp. 286–297, 2013.
- [4] A. Boubezoul, F. Dufour, S. Bouaziz, B. Larnaudie, and S. Espié, “Dataset on powered two wheelers fall and critical events detection,” *Data in Brief*, vol. 23, p. 103828, 2019.
- [5] S. Gelmini, S. Strada, M. Tanelli, S. Savaresi, and C. D. Tommasi, “Automatic crash detection system for two-wheeled vehicles: Design and experimental validation,” *IFAC-PapersOnLine*, vol. 52, no. 5, pp. 498–503, 2019.
- [6] S. Gelmini, S. C. Strada, M. Tanelli, S. M. Savaresi, and C. D. Tommasi, “A novel crash detection algorithm for two-wheeled vehicles,” *IEEE Transactions on Intelligent Vehicles*, vol. 6, no. 1, pp. 88–99, 2021.
- [7] A. J. Mahariba, A. U. R., and G. B. Rajan, “An efficient automatic accident detection system using inertial measurement through machine learning techniques for powered two wheelers,” *Expert Systems with Applications*, vol. 192, p. 116389, 2022.

- [8] S. Lee, B. Koo, and Y. Kim, “Pre-impact fall detection for e-scooter riding using an imu: Threshold-based, supervised, and unsupervised approaches,” *Applied Sciences*, vol. 14, no. 22, p. 10443, 2024.
- [9] T. Bianconcini, H. P. Monteagudo, A. Pjetri, T. Trinci, and L. Taccari, “VZCrash: A large-scale IMU dataset of ego-vehicle crashes,” arXiv:2606.06074, 2026. [Online]. Available: <https://arxiv.org/abs/2606.06074>
- [10] Blynk, “Events and device HTTPS api documentation,” 2026, accessed 28 August 2026. [Online]. Available: <https://docs.blynk.io/en/blynk.cloud/device-https-api/trigger-events-api>
- [11] TDK InvenSense, *MPU-6000 and MPU-6050 Product Specification, Revision 3.4*, 2013. [Online]. Available: <https://invensense.tdk.com/wp-content/uploads/2015/02/MPU-6000-Datasheet.pdf>
- [12] Espressif Systems, *ESP32 Series Datasheet*, 2025. [Online]. Available: https://www.espressif.com/sites/default/files/documentation/esp32_datasheet_en.pdf
- [13] SDSU HealthLINK Center, “Falls dataset archive,” 2026, downloaded 28 August 2026. [Online]. Available: <https://osf.io/n27yz/>
- [14] Wokwi, “MPU6050 simulator and automation reference,” 2026, accessed 28 August 2026. [Online]. Available: <https://docs.wokwi.com/parts/wokwi-mpu6050>
- [15] —, “Diagram file format reference,” 2026, accessed 28 August 2026. [Online]. Available: <https://docs.wokwi.com/diagram-format>
- [16] —, “Automation scenarios,” 2026, accessed 28 August 2026. [Online]. Available: <https://docs.wokwi.com/wokwi-ci/automation-scenarios>